HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN

IOT VÀ ỨNG DỤNG

Sinh viên thực hiện : **Đào Viết Trọng**
Mã sinh viên : **B23DCCN846**
Giảng viên hướng dẫn : **TS. Nguyễn Quốc Uy**

Hà Nội - 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI 3
1.1. Bối cảnh và lý do thực hiện 3
1.2. Mục tiêu của hệ thống 3
1.3. Yêu cầu phi chức năng (Ràng buộc kỹ thuật) 3
1.4. Công nghệ được sử dụng 3
CHƯƠNG 2: THIẾT KẾ TỔNG QUAN 4
2.1. Kiến trúc tổng thể 4
2.2. Thiết kế các lớp kiến trúc 5
2.3. Thiết kế phần cứng 5
2.4. Thiết kế phần mềm 6
2.5. Thiết kế cơ sở dữ liệu 6
2.5.1. Sơ đồ quan hệ 7
2.5.2. Bảng users 7
2.5.3. Bảng sensors 7
2.5.4. Bảng devices 8
2.5.5. Bảng sensor_data 8
2.5.6. Bảng action_history 8
2.5.7. Quy tắc dữ liệu quan trọng 9
2.6. Thiết kế luồng dữ liệu và giao tiếp 9
2.6.1. Luồng quan sát 9
2.6.2. Luồng điều khiển bóng đèn 9
2.6.3. Luồng tra cứu dữ liệu 10
2.7. MQTT Topic sử dụng 11
2.8. Nguyên tắc thiết kế và khả năng mở rộng 11
CHƯƠNG 3. THIẾT KẾ USE CASE 12
3.1. Use case: Đăng nhập 12
3.1.1. Giao diện 12
3.1.2. Đặc tả use case 13
3.1.3. Sequence diagram 14
3.1.4. Mô tả API 14
3.1.5. Mapping dữ liệu API – Database 15

3.2. Use case: Xem dữ liệu cảm biến thời gian thực 15
3.2.1. Giao diện 15
3.2.2. Đặc tả use case 15
3.2.3. Sequence diagram 16
3.2.4. Mô tả API 17
3.2.5. Mapping dữ liệu API – Database 17
3.3. Use case: Xem biểu đồ dữ liệu cảm biến 17
3.3.1. Giao diện 17
3.3.2. Đặc tả use case 18
3.3.3. Sequence diagram 18
3.3.4. Mô tả API 19
3.3.5. Mapping dữ liệu API – Database 19
3.4. Use case: Điều khiển bật/tắt thiết bị 20
3.4.1. Giao diện 20
3.4.2. Đặc tả use case 20
3.4.3. Sequence diagram 21
3.4.4. Mô tả API 22
3.4.5. Mapping dữ liệu API – Database 22
3.5. Use case: Xem lịch xử giá trị cảm biến 23
3.5.1. Giao diện 23
3.5.2. Đặc tả use case 23
3.5.3. Sequence diagram 24
3.5.4. Mô tả API 24
3.5.5. Mapping dữ liệu API – Database 24
3.6. Use case: Xem lịch sử điều khiển thiết bị 25
3.6.1. Giao diện 25
3.6.2. Đặc tả use case 25
3.6.3. Sequence diagram 26
3.6.4. Mô tả API 26
3.6.5. Mapping dữ liệu API – Database 27

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và lý do thực hiện

Với sự phát triển mạnh mẽ của Internet vạn vật (IoT), các hệ thống nhà thông minh (Smart Home) không chỉ giúp con người theo dõi môi trường sống mà còn tối ưu hóa việc điều khiển các thiết bị điện. Đề tài này được thực hiện nhằm ứng dụng các công nghệ IoT để giải quyết một bài toán thực tế: Xây dựng một trạm thu thập dữ liệu môi trường, đồng thời cho phép tương tác hai chiều (thu thập và điều khiển) thông qua mạng Internet.

1.2. Mục tiêu của hệ thống

Hệ thống được xây dựng hướng tới các mục tiêu cụ thể sau:

- Thu thập dữ liệu: Đo lường các chỉ số nhiệt độ, độ ẩm (từ cảm biến DHT11) và mức độ ánh sáng (từ cảm biến quang trở).
- Điều khiển thiết bị: Cho phép người dùng bật/tắt 2 bóng đèn từ xa thông qua giao diện Web.
- Lưu trữ dữ liệu: Lưu lại lịch sử các thông số môi trường vào cơ sở dữ liệu MySQL để phục vụ việc thống kê.
- Giám sát thời gian thực: Dữ liệu hiển thị trên Web App phải được cập nhật liên tục (Real-time).

1.3. Yêu cầu phi chức năng (Ràng buộc kỹ thuật)

- Tính song song (Non-blocking): Yêu cầu quan trọng nhất của hệ thống là việc điều khiển bóng đèn và việc đọc dữ liệu cảm biến phải hoạt động hoàn toàn độc lập. Lệnh bật/tắt đèn phải được thực thi ngay lập tức mà không bị trễ hay gián đoạn bởi chu kỳ đọc của cảm biến.
- Giao thức truyền thông: Sử dụng MQTT để đảm bảo tốc độ truyền tải nhanh, nhẹ và phù hợp với thiết bị vi điều khiển.

1.4. Công nghệ được sử dụng

Nhóm	Thành phần
Phần cứng	ESP32, DHT11, cảm biến ánh sáng(LDR),LED, điện trở và dây nối
Frontend	HTML, CSS, JavaScripts
Backend	Java(SpringBoot)
Giao tiếp web	REST API, WebSocket
Giao tiếp thiết bị	MQTT
MQTT Broker	Eclipse Mosquitto
Cơ sở dữ liệu	MySQL

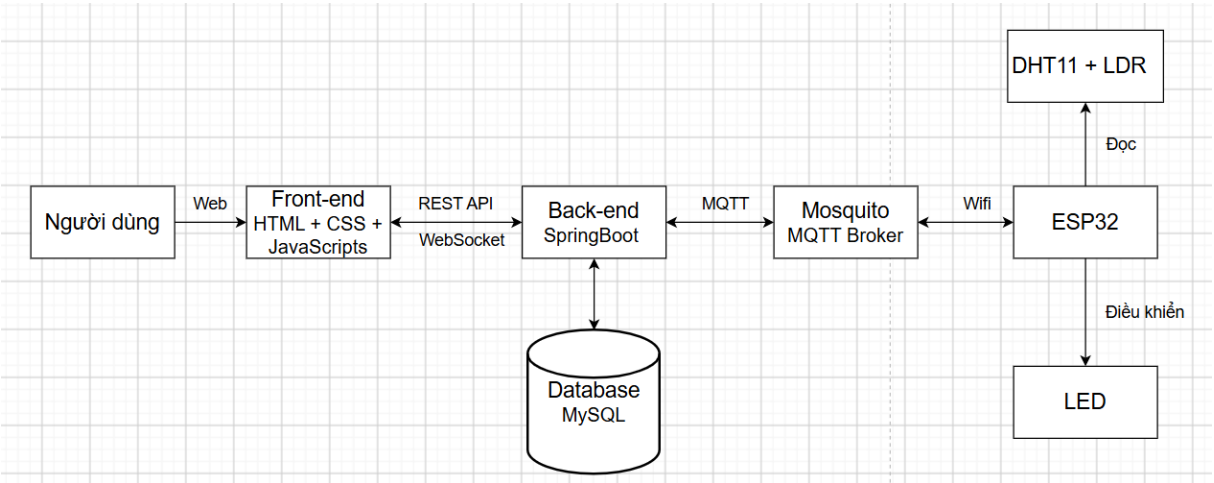
CHƯƠNG 2: THIẾT KẾ TỔNG QUAN

2.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình nhiều lớp, trong đó ESP32 đóng vai trò là thiết bị IoT trung tâm thực hiện thu thập dữ liệu từ cảm biến và điều khiển các thiết bị chấp hành. Dữ liệu từ ESP32 được truyền qua mạng Wi-Fi bằng giao thức MQTT đến MQTT Broker Eclipse Mosquitto. Backend Spring Boot kết nối với MQTT Broker để nhận dữ liệu cảm biến, xử lý nghiệp vụ và lưu trữ dữ liệu vào cơ sở dữ liệu MySQL.

Phía giao diện web sử dụng Thymeleaf kết hợp với HTML, CSS và JavaScript. Bootstrap được sử dụng để xây dựng giao diện, trong khi Chart.js được sử dụng để biểu diễn dữ liệu cảm biến dưới dạng biểu đồ. WebSocket kết hợp với STOMP được sử dụng để truyền dữ liệu mới từ backend đến giao diện theo thời gian thực mà không cần người dùng tải lại trang.

Đối với chức năng điều khiển, người dùng thực hiện thao tác bật/tắt hai bóng đèn trên giao diện web. Yêu cầu điều khiển được gửi đến Spring Boot, sau đó backend publish lệnh điều khiển lên MQTT Broker. ESP32 subscribe các topic điều khiển tương ứng, thực hiện bật/tắt LED và phản hồi trạng thái thiết bị về backend. Chức năng điều khiển bóng đèn được tách biệt với quá trình thu thập dữ liệu của DHT11 và cảm biến quang trở, do đó việc bật/tắt bóng đèn không làm ảnh hưởng đến hoạt động của các cảm biến.



Hình 2.1. Kiến trúc tổng quan của hệ thống

2.2. Thiết kế các lớp kiến trúc

Lớp	Thành Phần	Trách nhiệm chính
Thiết bị	ESP32, DHT11, LDR, 2 bóng đèn led	Thu thập nhiệt độ, độ ẩm, ánh sáng và thực thi bật/tắt bóng đèn
Giao tiếp Iot	Wifi, MQTT, Eclipse Mosquitto	Truyền dữ liệu cảm biến, lệnh điều khiển và trạng thái thiết bị theo mô hình Publish/Subscribe
Ứng dụng	Spring Boot, Spring Web, MQTT Client, WebSocket/STOMP	Tiếp nhận dữ liệu MQTT, xử lý nghiệp vụ, cung cấp REST API, truyền dữ liệu realtime và gửi lệnh điều khiển
Dữ liệu	MySQL, Spring Data JPA, Hibernate	Lưu trữ thông tin người dùng, cảm biến, thiết bị, dữ liệu đo và lịch sử điều khiển
Trình bày	HTML, CSS, JavaScript, Bootstrap, Chart.js	Hiển thị Dashboard, dữ liệu cảm biến, biểu đồ, lịch sử và giao diện điều khiển thiết bị

2.3. Thiết kế phần cứng

ESP32 là bộ vi điều khiển trung tâm của hệ thống, có khả năng kết nối Wi-Fi và hỗ trợ nhiều chân GPIO, phù hợp cho việc kết nối các cảm biến và thiết bị chấp hành.

DHT11 được sử dụng để đo hai đại lượng môi trường là nhiệt độ và độ ẩm. Cảm biến quang trở LDR được sử dụng để xác định mức độ ánh sáng của môi trường thông qua giá trị đọc từ đầu vào analog của ESP32.

Hai bóng đèn được mô phỏng bằng hai LED, mỗi LED được nối với một chân GPIO riêng của ESP32 thông qua một điện trở hạn dòng. Hai LED hoạt động như hai thiết bị chấp hành độc lập, tương ứng với bóng đèn 1 và bóng đèn 2.

Hệ thống phần cứng được chia thành hai nhóm chức năng độc lập:

- Nhóm cảm biến**:** DHT11 và LDR, thực hiện việc thu thập dữ liệu môi trường theo chu kỳ.
- Nhóm thiết bị chấp hành: LED 1 và LED 2 điều khiển bóng đèn 1 và bóng đèn 2.

Việc tách biệt hai nhóm giúp bảo đảm rằng thao tác bật hoặc tắt bóng đèn không làm dừng hoặc thay đổi quá trình đọc dữ liệu từ DHT11 và LDR.

ESP32 đồng thời đảm nhiệm hai cơ chế MQTT:

- Publish: gửi dữ liệu nhiệt độ, độ ẩm và ánh sáng lên MQTT Broker.
- Subscribe: nhận lệnh bật/tắt bóng đèn từ backend thông qua các topic điều khiển tương ứng.

Sau khi thực hiện lệnh, ESP32 có thể publish lại trạng thái thực tế của bóng đèn để backend cập nhật trạng thái trên hệ thống.

2.4. Thiết kế phần mềm

Thành phần	Xử lý chính	Đầu ra
Firmware ESP32	Đọc DHT11, đọc LDR, kết nối Wi-Fi, publish dữ liệu MQTT, subscribe lệnh điều khiển LED	Dữ liệu cảm biến và trạng thái bóng đèn
MQTT Broker	Nhận và chuyển tiếp message giữa ESP32 và backend	Message MQTT
Spring Boot	Nhận dữ liệu MQTT, kiểm tra và xử lý dữ liệu, lưu MySQL, cung cấp REST API, WebSocket/STOMP và publish lệnh MQTT	Dữ liệu database, JSON API, dữ liệu realtime và lệnh MQTT
MySQL	Lưu dữ liệu cảm biến, trạng thái thiết bị và lịch sử điều khiển	Dữ liệu hiện tại và dữ liệu lịch sử
JavaScript	Hiển thị dữ liệu, gọi REST API, nhận dữ liệu WebSocket và gửi yêu cầu điều khiển	Dashboard, biểu đồ, bảng lịch sử và trạng thái thiết bị
Bootstrap	Xây dựng bố cục và các thành phần giao diện	Giao diện web responsive
Chart.js	Biểu diễn nhiệt độ, độ ẩm và ánh sáng theo thời gian	Biểu đồ dữ liệu cảm biến

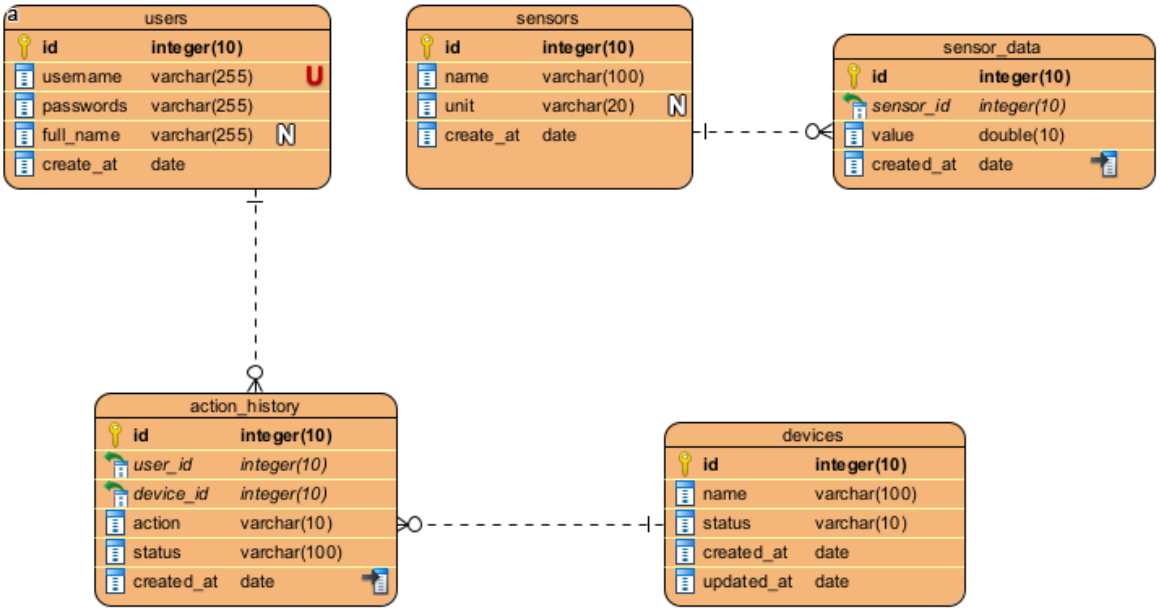
2.5. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MySQL để lưu trữ dữ liệu lâu dài. Cơ sở dữ liệu gồm các bảng chính: users, sensors, devices, sensor_data và action_history.

Bảng sensor_data lưu các giá trị nhiệt độ, độ ẩm và ánh sáng cùng thời điểm đo. Bảng devices lưu trạng thái hiện tại của hai bóng đèn. Bảng action_history lưu lại các thao tác bật/tắt do người dùng thực hiện.

Việc lưu riêng dữ liệu cảm biến và lịch sử điều khiển giúp hệ thống có thể truy vấn độc lập dữ liệu môi trường và các thao tác điều khiển thiết bị.

2.5.1. Sơ đồ quan hệ



Hình 2.2. Sơ đồ quan hệ dữ liệu

2.5.2. Bảng users

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Mã người dùng
username	VARCHAR(255)	UNIQUE, NOT NULL	Tên đăng nhập
password	VARCHAR(255)	NOT NULL	Mật khẩu đã băm
full_name	VARCHAR(150)	NULL	Họ tên người dùng
created_at	DATETIME	NOT NULL	Thời điểm tạo tài khoản

2.5.3. Bảng sensors

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Mã cảm biến
name	VARCHAR(100)	NOT NULL	Tên cảm biến
unit	VARCHAR(20)	NULL	Đơn vị đo
created_at	DATETIME	NOT NULL	Thời điểm tạo cảm biến

2.5.4. Bảng devices

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Mã thiết bị
name	VARCHAR(100)	NOT NULL	Tên thiết bị
status	ENUM(‘ON’, ‘OFF’)	NOT NULL	Trạng thái hiện tại
created_at	DATETIME	NOT NULL	Thời điểm tạo thiết bị
updated_at	DATETIME	NOT NULL	Thời điểm cập nhật trạng thái

2.5.5. Bảng sensor_data

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Mã bản ghi
sensor_id	INT	FK → sensors.id, NOT NULL	Mã cảm biến
value	DOUBLE	NOT NULL	Giá trị đo
created_at	DATETIME	NOT NULL, INDEX	Thời điểm ghi nhận

2.5.6. Bảng action_history

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Mã lịch sử
user_id	INT	FK → users.id, NOT NULL	Người thực hiện
device_id	INT	FK → devices.id, NOT NULL	Thiết bị được điều khiển
action	ENUM(‘ON’, ‘OFF’)	NOT NULL	Lệnh người dùng yêu cầu
status	ENUM(‘PENDING’, ‘SUCCESS’, ‘FAILED’, ‘TIMEOUT’)	NOT NULL	Kết quả xử lý lệnh

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
created_at	DATETIME	NOT NULL, INDEX	Thời điểm thực hiện

2.5.7. Quy tắc dữ liệu quan trọng

- devices.status lưu trạng thái hiện tại ON/OFF của bóng đèn.
- action_history.action lưu lệnh ON/OFF do người dùng yêu cầu.
- Khi backend vừa gửi lệnh điều khiển, action_history.status được đặt là PENDING.
- Khi ESP32 xác nhận đã thực hiện lệnh, , action_history.status được đặt là SUCCESS và devices.status được cập nhật tương ứng là ON hoặc OFF.
- Nếu không nhận được phản hồi trong khoảng thời gian quy định, yêu cầu được đánh dấu TIMEOUT và giữ nguyên devices.status cuối cùng đã được xác nhận.
- Mỗi bản ghi sensor_data liên kết với đúng một cảm biến.
- Một cảm biến có thể có nhiều bản ghi dữ liệu theo thời gian.
- Mật khẩu người dùng không được lưu dưới dạng văn bản rõ; phải mã hóa mật khẩu trước khi lưu và so sánh bằng thuật toán.
- Các trường thời gian và khóa ngoại cần được đánh chỉ mục để hỗ trợ truy vấn dữ liệu lịch sử.

2.6. Thiết kế luồng dữ liệu và giao tiếp

2.6.1. Luồng quan sát

Luồng giám sát được thực hiện theo trình tự:

→ ESP32 đọc dữ liệu từ cảm biến DHT11 và LDR

→ Xử lý, kiểm tra và đóng gói dữ liệu

→ Publish dữ liệu lên MQTT Broker Mosquitto

→ Spring Boot subscribe, tiếp nhận và phân tích dữ liệu JSON

→ Lưu dữ liệu vào cơ sở dữ liệu MySQL

→ Phát dữ liệu mới qua WebSocket/STOMP

→ Giao diện Web App nhận dữ liệu và cập nhật Dashboard.

2.6.2. Luồng điều khiển bóng đèn

Luồng điều khiển được thực hiện theo trình tự:

Giao diện web gọi API: GET /api/devices/{deviceId}/status để lấy trạng thái cuối cùng đã được backend xác nhận của đèn. Thao tác điều khiển được cho phép khi trạng thái đã được tải thành công.

→ Người dùng gửi yêu cầu ON/OFF.

→ Backend tạo action_history với trạng thái PENDING.

→ Backend publish lệnh điều khiển lên MQTT Broker thông qua topic tương ứng với bóng đèn:

- iot/device01/control/lamp1
- iot/device01/control/lamp2

→ ESP32 subscribe topic điều khiển, nhận lệnh ON hoặc OFF và thay đổi trạng thái LED tương ứng.

→ Sau khi thực hiện lệnh, ESP32 publish trạng thái hiện tại của LED về backend thông qua MQTT.

→ ESP32 publish trạng thái lên MQTT Broker thông qua topic tương ứng:

- iot/device01/status/lamp1
- iot/device01/status/lamp2

→ Backend subscribe topic trạng thái và thực hiện các thao tác sau:

- Cập nhật action_history.status từ PENDING thành SUCCESS.
- Cập nhật devices.status thành ON hoặc OFF.
- Cập nhật devices.updated_at và gửi trạng thái mới đến giao diện thông qua WebSocket/STOMP.

Nếu sau thời gian cố định kể từ thời điểm publish lệnh MQTT mà backend không nhận được phản hồi từ ESP32 (TIMEOUT), backend thực hiện các thao tác:

- Cập nhật action_history.status từ PENDING thành TIMEOUT.
- Giữ nguyên devices.status là trạng thái cuối cùng đã được ESP32 xác nhận.
- Gửi thông báo TIMEOUT đến giao diện web thông qua WebSocket/STOMP để hiển thị trạng thái không xác định; không kết luận rằng bóng đèn đang ON hoặc OFF.
- Khi ESP32 kết nối MQTT trở lại, ESP32 publish lại trạng thái hiện tại của các LED lên các topic status. Backend cập nhật devices.status và gửi trạng thái mới đến giao diện qua WebSocket/STOMP.

2.6.3. Luồng tra cứu dữ liệu

Luồng tra cứu dữ liệu được thực hiện theo trình tự :

Web App gửi REST API cùng điều kiện lọc

→ Backend gọi xuống JPA/Hibernate

→ Truy vấn MySQL

→ Giao diện hiển thị dữ liệu được trả về

2.7. MQTT Topic sử dụng

Hệ thống sử dụng các topic MQTT riêng biệt cho dữ liệu cảm biến, lệnh điều khiển và trạng thái thiết bị.

Topic	Publisher	Subscriber	Chức năng
iot/device01/sensor	ESP32	Backend	Gửi nhiệt độ, độ ẩm và ánh sáng
iot/device01/control/lamp1	Backend	ESP32	Gửi lệnh ON/OFF cho bóng đèn 1
iot/device01/control/lamp2	Backend	ESP32	Gửi lệnh ON/OFF cho bóng đèn 2
iot/device01/status/lamp1	ESP32	Backend	Gửi trạng thái của bóng đèn 1
iot/device01/status/lamp2	ESP32	Backend	Gửi trạng thái của bóng đèn 2

2.8. Nguyên tắc thiết kế và khả năng mở rộng

Hệ thống được xây dựng dựa trên một số nguyên tắc chính:

- ESP32 chịu trách nhiệm giao tiếp với cảm biến và thiết bị chấp hành.
- MQTT đảm nhiệm việc truyền thông giữa ESP32 và backend theo mô hình Publish/Subscribe.
- Spring Boot chịu trách nhiệm xử lý nghiệp vụ, quản lý dữ liệu, cung cấp REST API và WebSocket/STOMP.
- MySQL chịu trách nhiệm lưu trữ dữ liệu lâu dài.
- Thymeleaf, JavaScript và Bootstrap chịu trách nhiệm xây dựng giao diện web.
- Chart.js được sử dụng để biểu diễn dữ liệu cảm biến trực quan.
- Dữ liệu realtime được truyền bằng WebSocket/STOMP để giảm nhu cầu gửi request liên tục từ trình duyệt.
- Chức năng đọc cảm biến và điều khiển bóng đèn được tách biệt ở cả phần cứng, firmware và MQTT topic.
- Có thể mở rộng hệ thống bằng cách bổ sung cảm biến hoặc thiết bị mới, đồng thời tạo các topic MQTT tương ứng mà không cần thay đổi toàn bộ kiến trúc.
- Các thông tin cấu hình như địa chỉ MQTT Broker, thông tin kết nối MySQL và tài khoản MQTT cần được tách khỏi mã nguồn.

- Khi triển khai trong môi trường thực tế cần bổ sung cơ chế xác thực, phân quyền, HTTPS/WSS và xác thực MQTT.

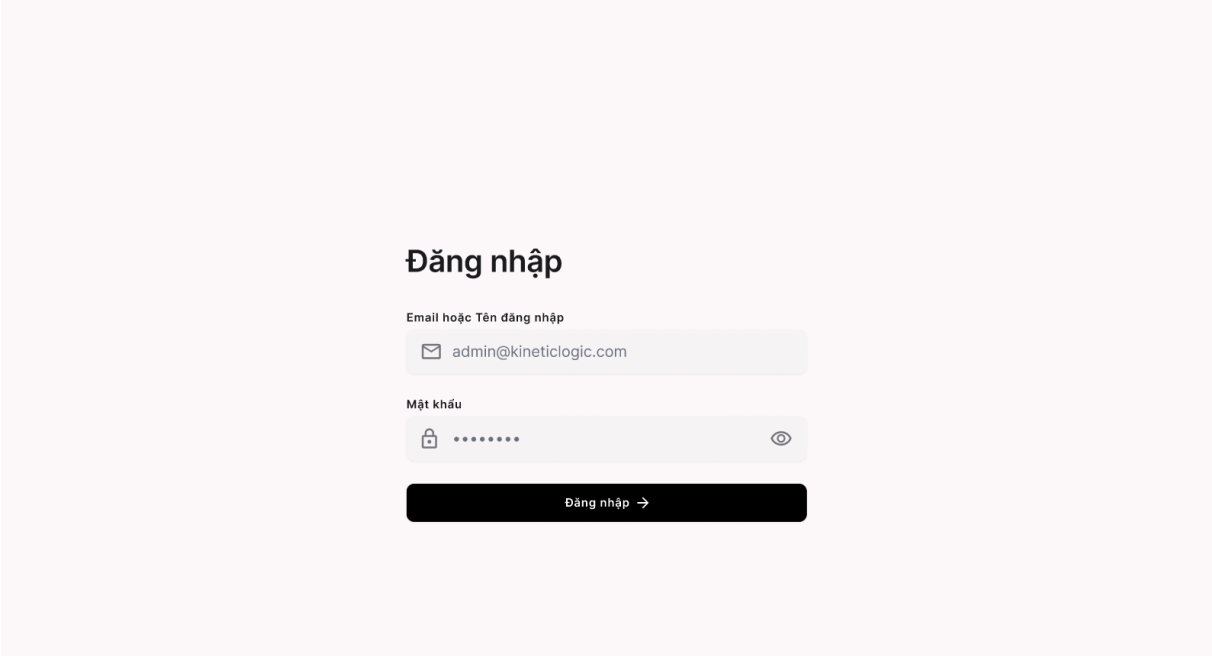
CHƯƠNG 3. THIẾT KẾ USE CASE

Hệ thống gồm 6 use case chính. Các chức năng: theo dõi dữ liệu thời gian thực, xem biểu đồ và điều khiển hai bóng đèn được đặt trên màn hình Dashboard. Các chức năng tra cứu lịch sử dữ liệu cảm biến và lịch sử điều khiển được thực hiện trên các màn hình riêng.

Tên use case	Mô tả ngắn	Màn hình
Đăng nhập	Xác thực tài khoản	Login
Xem dữ liệu cảm biến thời gian thực	Hiển thị nhiệt độ, độ ẩm và ánh sáng mới nhất.	Dashboard
Xem biểu đồ dữ liệu cảm biến	Bểu diễn và cập nhật xu hướng nhiệt độ, độ ẩm và ánh sáng theo thời gian.	Dashboard
Điều khiển bật/tắt thiết bị	Gửi lệnh ON/OFF cho hai bóng đèn và theo dõi trạng thái phản hồi.	Dashboard
Xem lịch sử giá trị cảm biến	Lọc, sắp xếp và phân trang dữ liệu cảm biến đã lưu.	Sensor Data
Xem lịch sử điều khiển thiết bị	Lọc và xem các lần bật/tắt bóng đèn.	Device History

3.1. Use case: Đăng nhập

3.1.1. Giao diện

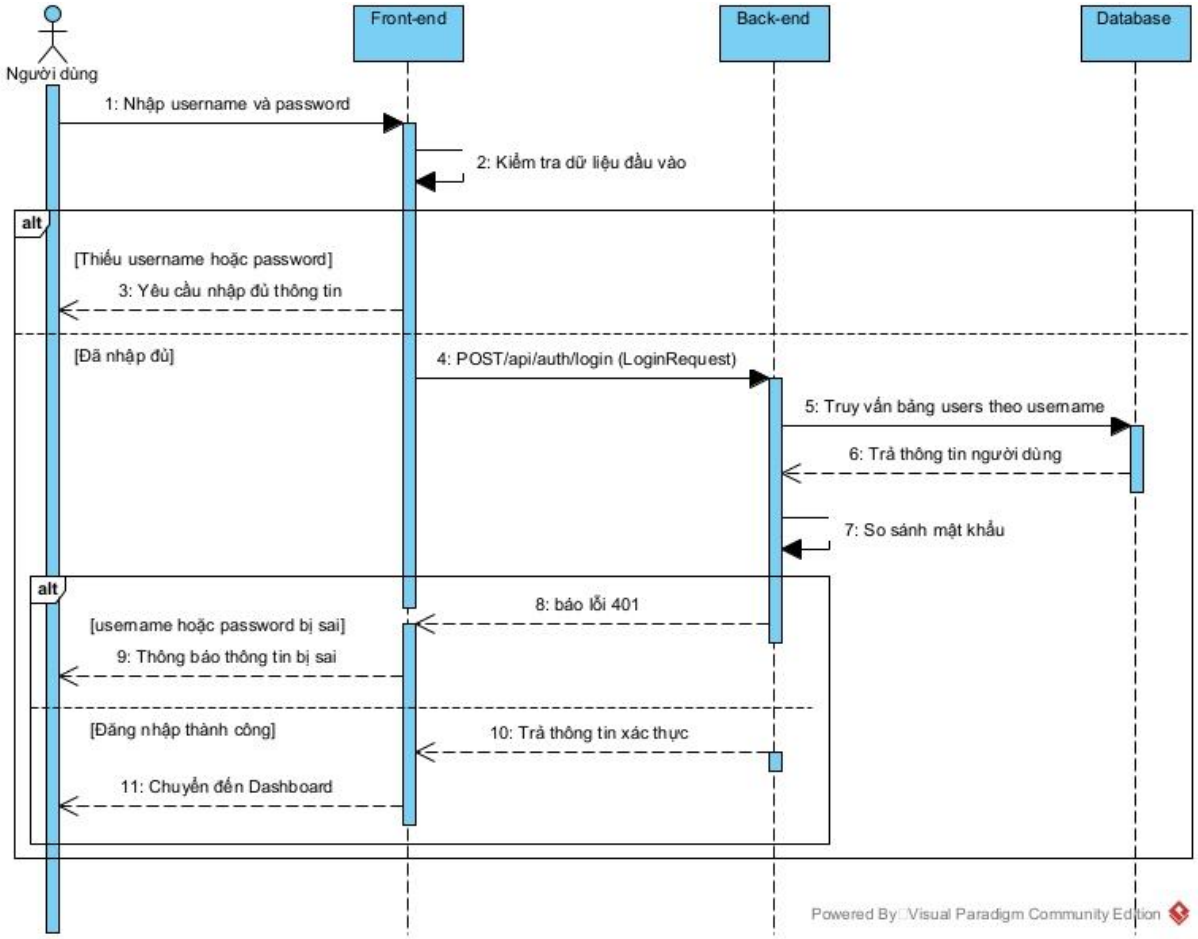


Hình 3.1. Màn hình đăng nhập

3.1.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Đăng nhập
Actor	Người dùng
Tiền điều kiện	Người dùng đã có tài khoản; Spring Boot và MySQL đang hoạt động
Hậu điều kiện	Người dùng được xác thực thành công và chuyển vào trong Dashboard
Luồng thực hiện	Bước 1: Người dùng nhập username và password. Bước 2: Giao diện gửi yêu cầu POST /api/auth/login đến Spring Boot. Bước 3: Spring Boot truy vấn bảng users trong MySQL. Bước 4: Backend kiểm tra thông tin xác thực. Bước 5: Nếu thông tin hợp lệ, backend trả kết quả xác thực. Bước 6: Giao diện chuyển người dùng đến Dashboard.
Luồng ngoại lệ	• Người dùng bỏ trống username hoặc password: giao diện yêu cầu nhập đầy đủ dữ liệu. • Username hoặc password không đúng: backend trả mã lỗi xác thực và giao diện hiển thị thông báo. • Không kết nối được MySQL: hệ thống thông báo lỗi máy chủ.

3.1.3. Sequence diagram



Hình 3.2. Sequence diagram – Đăng nhập

3.1.4. Mô tả API

Thuộc tính	Nội dung
Tên API	Đăng nhập
Method	POST
URL	/api/auth/login
Params / Query	Không có

Body	{
	“username” : “abc”,
	“password” : “123”
	}

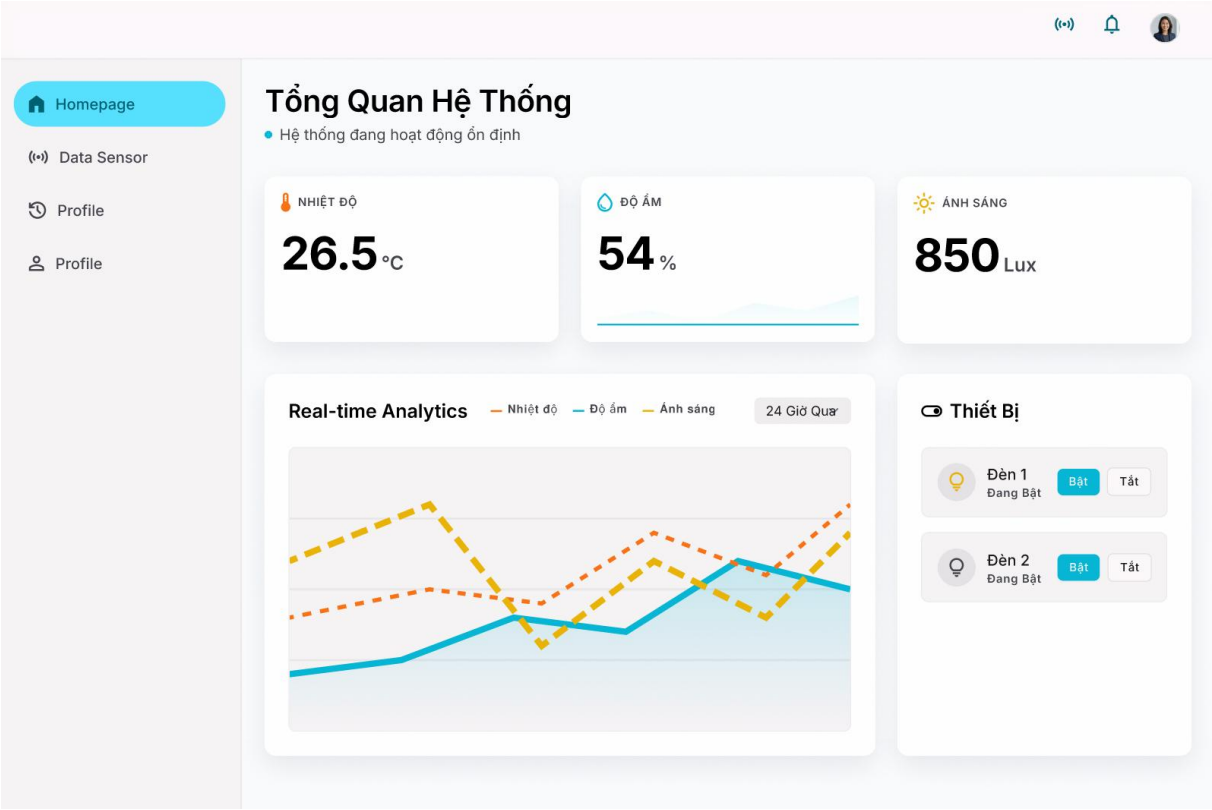
Response	{
	“id” : 1,
	“username” : “abc”,
	“full_name” : “Nguyen Van A”
	}

3.1.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
id	users	id
username	users	username
full_name	users	full_name

3.2. Use case: Xem dữ liệu cảm biến thời gian thực

3.2.1. Giao diện



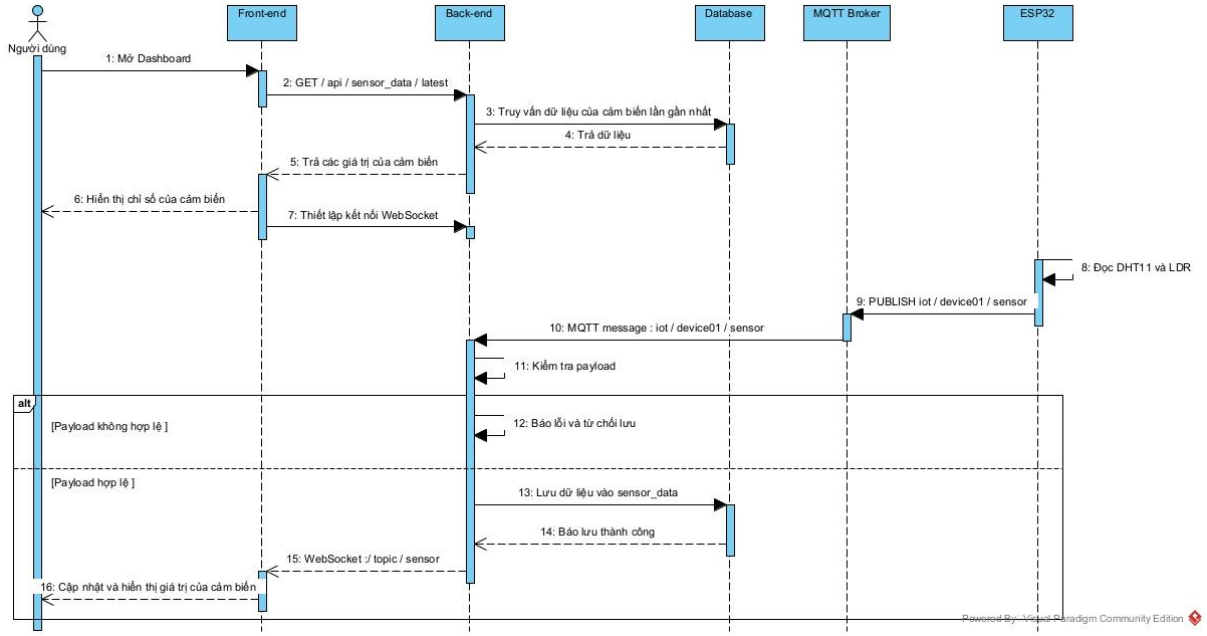
Hình 3.3. Giao diện dashboard – dữ liệu cảm biến thời gian thực

3.2.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Xem dữ liệu cảm biến thời gian thực
Actor	Người dùng đã được xác thực
Tiền điều kiện	Người dùng đã đăng nhập; ESP32, Mosquitto, Spring Boot, MySQL và Web Application đang hoạt động.
Hậu điều kiện	Dashboard hiển thị dữ liệu cảm biến gần nhất và tự động cập nhật khi có dữ liệu mới.
Luồng thực hiện	Bước 1: Người dùng mở Dashboard. Bước 2: JavaScript gọi GET /api/sensor-data/latest để lấy dữ liệu hiện tại. Bước 3: Web Application thiết lập kết nối WebSocket/STOMP với Spring Boot.

	Bước 4: ESP32 đọc nhiệt độ, độ ẩm từ DHT11 và ánh sáng từ LDR. Bước 5: ESP32 đóng gói dữ liệu và publish lên topic: iot/device01/sensor Bước 6: Mosquitto chuyển message đến Spring Boot. Bước 7: Spring Boot phân tích dữ liệu MQTT và lưu dữ liệu vào sensor_data ở Database. Bước 8: Spring Boot gửi dữ liệu mới tới Web Application thông qua WebSocket/STOMP. Bước 9: JavaScript cập nhật các giá trị nhiệt độ, độ ẩm và ánh sáng trên Dashboard mà không cần tải lại trang.
Luồng ngoại lệ	• ESP32 mất kết nối Wi-Fi/MQTT: giao diện giữ dữ liệu cuối cùng và hiển thị trạng thái thiết bị mất kết nối. • WebSocket mất kết nối: giao diện hiển thị trạng thái mất kết nối và thực hiện kết nối lại. • Payload MQTT không hợp lệ: Spring Boot từ chối lưu dữ liệu và ghi log lỗi. • MySQL không hoạt động: dữ liệu mới không được lưu, đồng thời backend ghi nhận lỗi hệ thống.

3.2.3. Sequence diagram



Hình 3.4. Sequence diagram – Xem dữ liệu cảm biến thời gian thực

3.2.4. Mô tả API

Thuộc tính	Nội dung
Tên API	Lấy dữ liệu cảm biến gần nhất
Method	GET
URL	/api/sensor-data/latest
Params / Query	Không có
Body	Không có

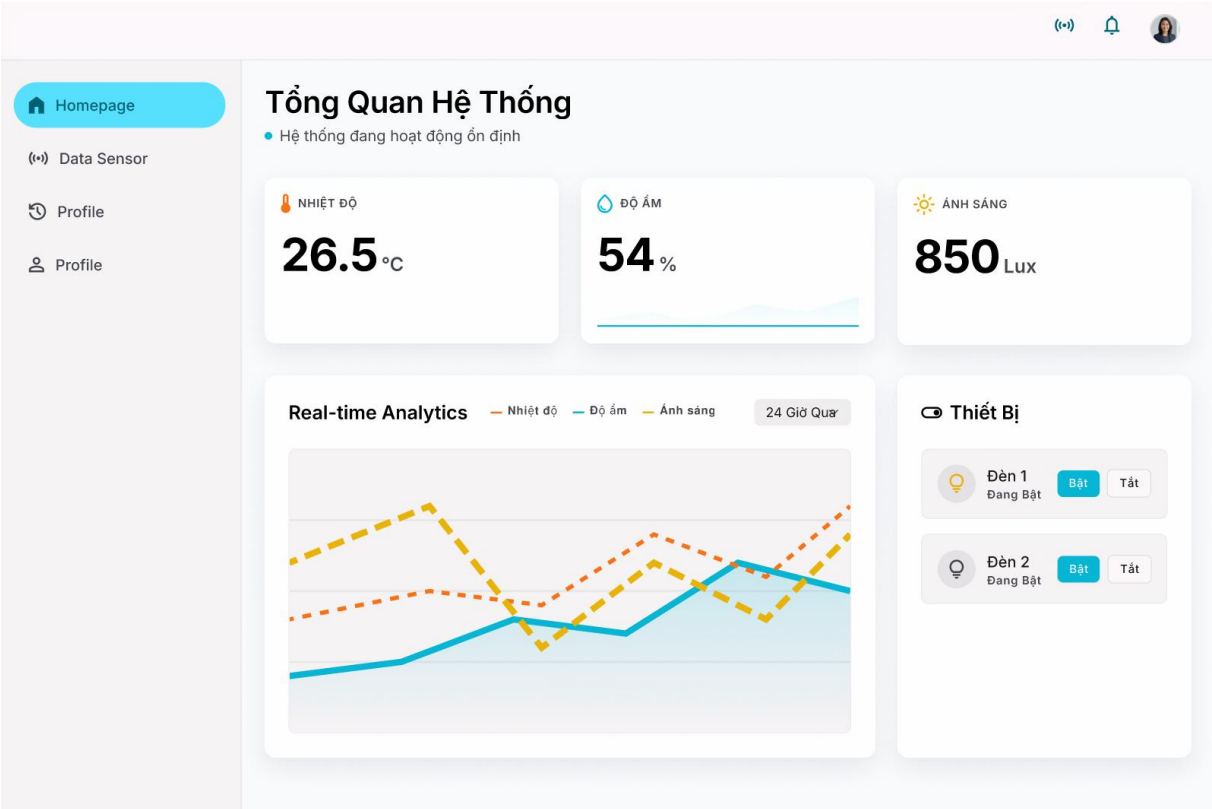
Response	{
	"temperature":28.5,
	"humidity":70.2,
	"light":1530,
	"time":"2026-08-19T10:20:05"
	}

3.2.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
temperature	sensor_data + sensors	value, name
humidity	sensor_data + sensors	value, name
light	sensor_data + sensors	value, name
time	sensor_data	created_at

3.3. Use case: Xem biểu đồ dữ liệu cảm biến

3.3.1. Giao diện

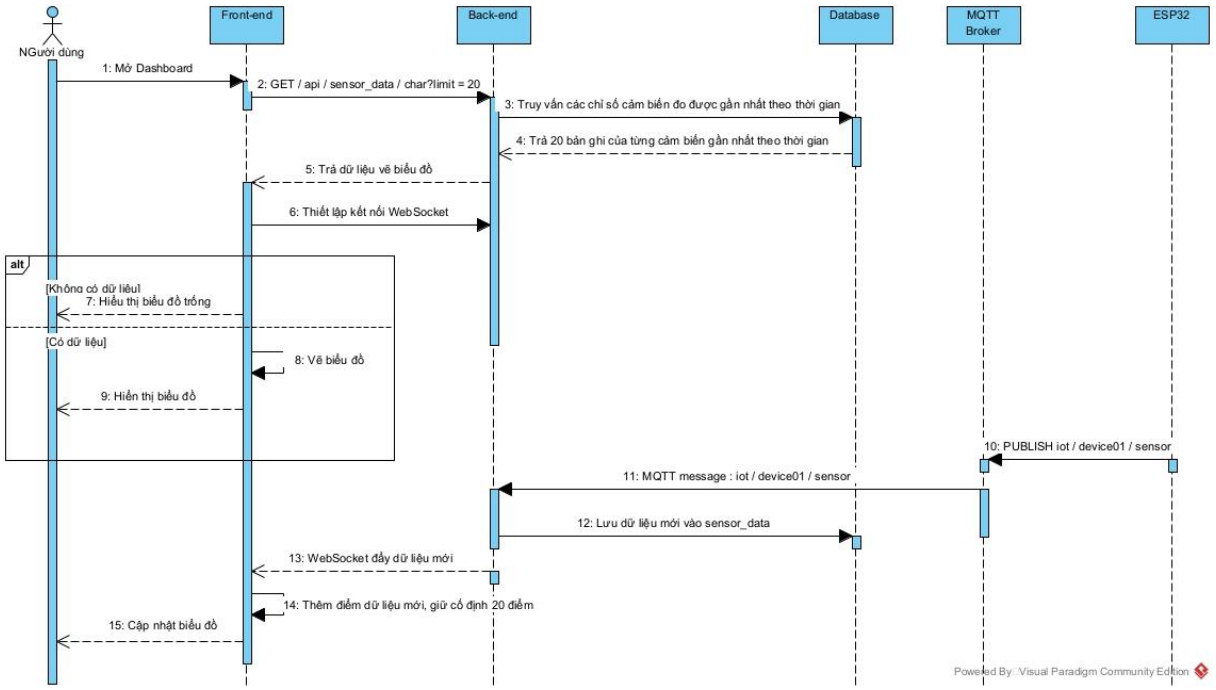


Hình 3.5. Dashboard – biểu đồ dữ liệu cảm biến

3.3.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Xem biểu đồ dữ liệu cảm biến
Actor	Người dùng đã được xác thực
Tiền điều kiện	Người dùng đã đăng nhập; MySQL có dữ liệu cảm biến hoặc hệ thống đang nhận dữ liệu mới.
Hậu điều kiện	Biểu đồ hiển thị dữ liệu theo thời gian và được cập nhật khi có giá trị mới.
Luồng thực hiện	Bước 1: Người dùng mở Dashboard. Bước 2: JavaScript gọi GET /api/sensor-data/chart để lấy dữ liệu gần nhất. Bước 3: Spring Boot truy vấn dữ liệu từ MySQL. Bước 4: Backend trả dữ liệu theo thời gian dưới dạng JSON. Bước 5: JavaScript sử dụng Chart.js để tạo biểu đồ. Bước 6: Khi Spring Boot nhận được dữ liệu cảm biến mới từ ESP32, backend phát dữ liệu qua WebSocket/STOMP. Bước 7: JavaScript nhận dữ liệu mới và thêm điểm mới vào biểu đồ.
Luồng ngoại lệ	• Không có dữ liệu: biểu đồ hiển thị trạng thái chưa có dữ liệu. • Không kết nối được WebSocket: biểu đồ giữ dữ liệu hiện tại và giao diện hiển thị trạng thái mất kết nối. • Dữ liệu không hợp lệ: JavaScript bỏ qua bản ghi không hợp lệ.

3.3.3. Sequence diagram



Hình 3.6. Sequence diagram – Xem biểu đồ dữ liệu cảm biến

3.3.4. Mô tả API

Thuộc tính	Nội dung
Tên API	Lấy dữ liệu biểu đồ
Method	GET
URL	/api/sensor-data/chart
Params / Query	limit=20
Body	Không có

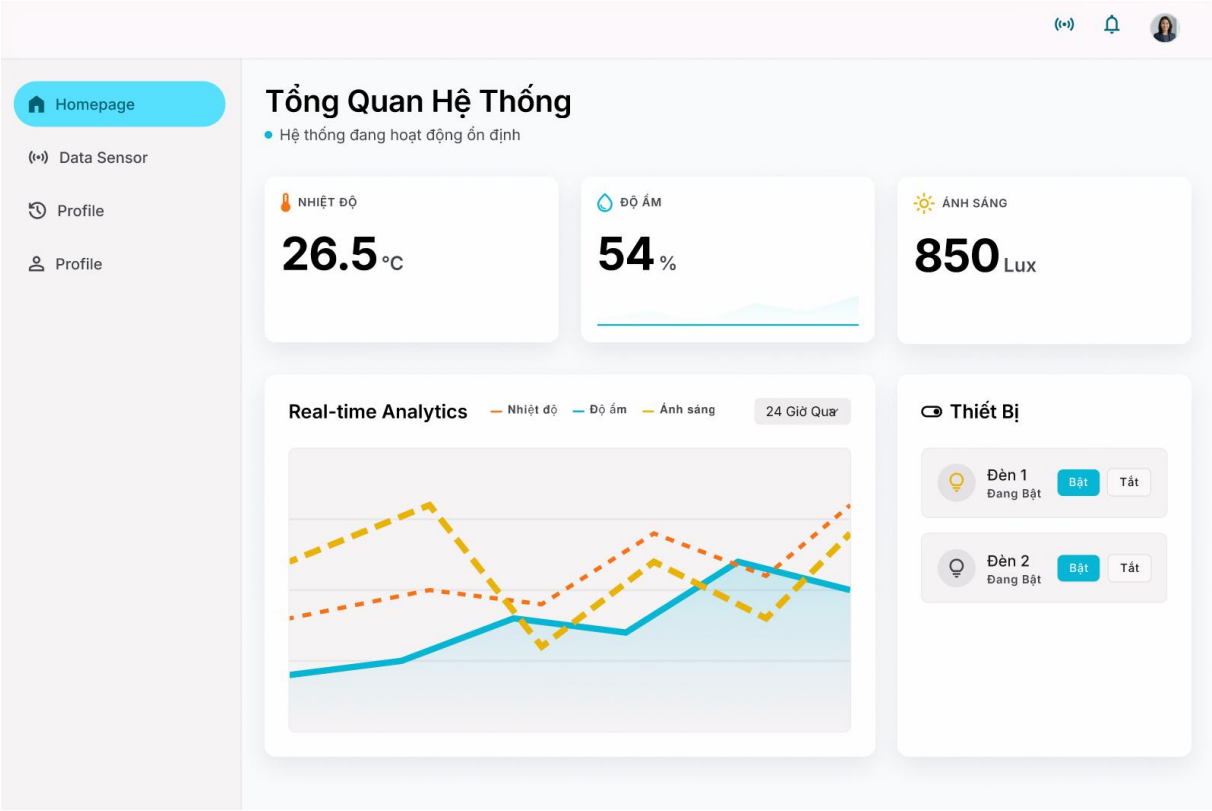
Response	{
	"temperature":
	[{"time":"10:00","value":28.1},
	{"time":"10:02","value":28.5}],
	"humidity":
	[{"time":"10:00","value":70},
	{"time":"10:02","value":70.2}],
	"light": [{"time":"10:00","value":1500},
	{"time":"10:02","value":1530}]
	}

3.3.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
temperature[].value	sensor_data	value
humidity[].value	sensor_data	value
light[].value	sensor_data	value
*.time	sensor_data	created_at
Tên chuỗi dữ liệu	sensors	name

3.4. Use case: Điều khiển bật/tắt thiết bị

3.4.1. Giao diện



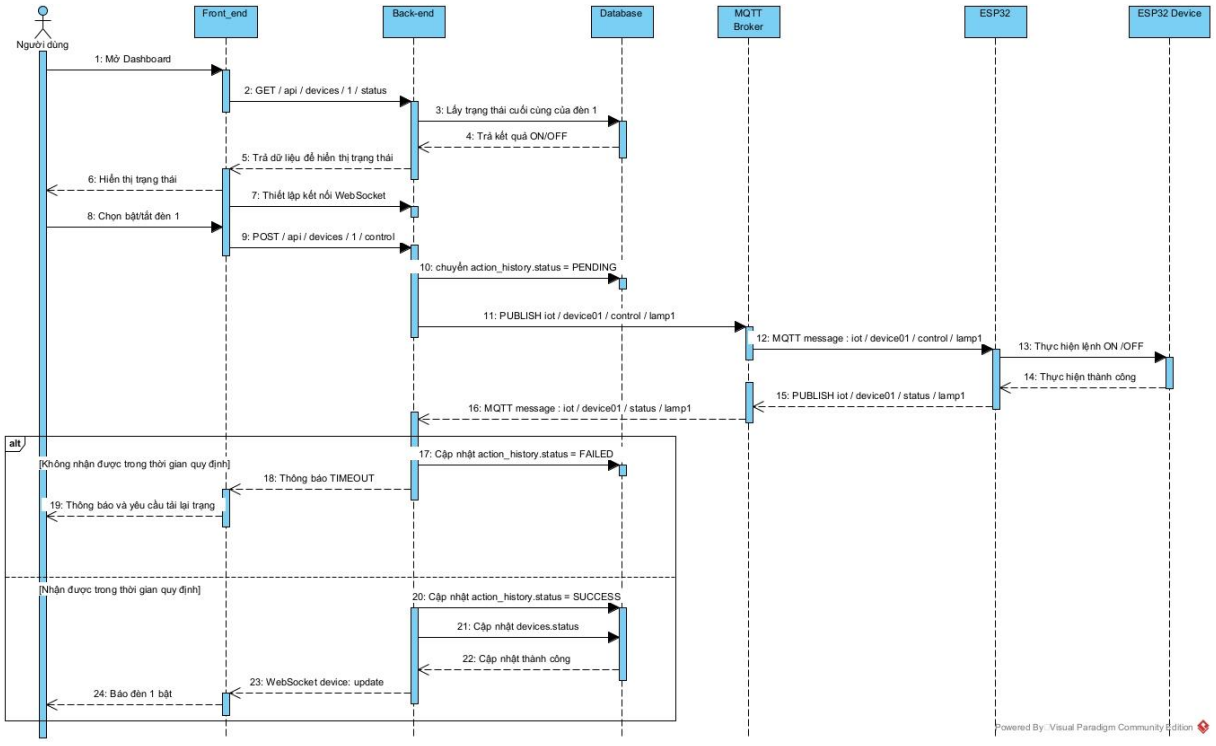
Hình 3.7. Dashboard – khu vực điều khiển thiết bị

3.4.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Điều khiển bật/tắt thiết bị
Actor	Người dùng đã được xác thực
Tiền điều kiện	Người dùng đã đăng nhập; thiết bị tồn tại; Spring Boot, Mosquitto và ESP32 đang hoạt động hoặc đã có trạng thái cuối cùng được lưu.
Hậu điều kiện	Lệnh được gửi đến thiết bị; trạng thái xác nhận của thiết bị được cập nhật nếu nhận được phản hồi từ ESP32.
Luồng thực hiện	Bước 1: Khi mở Dashboard, JavaScript gọi: GET /api/devices/{deviceId}/status để lấy trạng thái cuối cùng được backend xác nhận. Bước 2: Giao diện hiển thị trạng thái của đèn 1 và đèn 2. Bước 3: Người dùng chọn ON hoặc OFF. Bước 4: JavaScript gửi: POST /api/devices/{deviceId}/control đến Spring Boot. Bước 5: Backend kiểm tra thiết bị và tạo bản ghi action_history với trạng thái PENDING. Bước 6: Spring Boot publish lệnh MQTT. Đối với đèn 1:

	iot/device01/control/lamp1 Đối với đèn 2: iot/device01/control/lamp2 Bước 7: ESP32 nhận lệnh và điều khiển LED tương ứng. Bước 8: Sau khi thực hiện lệnh, ESP32 publish trạng thái hiện tại của LED. Đối với đèn 1: iot/device01/status/lamp1 Đối với đèn 2: iot/device01/status/lamp2 Bước 9: Spring Boot nhận trạng thái, cập nhật devices.status, cập nhật action_history.status thành SUCCESS và phát trạng thái mới đến Web Application qua WebSocket/STOMP. Bước 10: JavaScript cập nhật trạng thái bóng đèn trên Dashboard.
Luồng ngoại lệ	• Không lấy được trạng thái cuối: Khóa nút điều khiển và yêu cầu tải lại trang • Thiết bị không tồn tại: backend trả lỗi. • Lệnh không hợp lệ: backend trả lỗi HTTP 400. • Sau khoảng thời gian cố định (5s) kể từ lúc publish lệnh điều khiển mà không nhận được topic trạng thái: action_history chuyển TIMEOUT, devices.status giữ nguyên trạng thái cuối cùng đã được xác nhận; giao diện hiển thị trạng thái không xác định và thông báo TIMEOUT.

3.4.3. Sequence diagram



Hình 3.8. Sequence diagram – Điều khiển bật/tắt thiết bị

3.4.4. Mô tả API

Thuộc tính	Nội dung
Tên API	Lấy trạng thái thiết bị
Method	GET
URL	/api/devices/{deviceId}/status
Params / Query	deviceId
Body	Không có

Response	{
	“deviceId”:1,
	“status” : OFF,
	}

Thuộc tính	Nội dung
Tên API	Điều khiển thiết bị
Method	POST
URL	/api/devices/{deviceId}/control
Params / Query	deviceId

Body	{
	“deviceId”:1,
	“action”:ON
	}

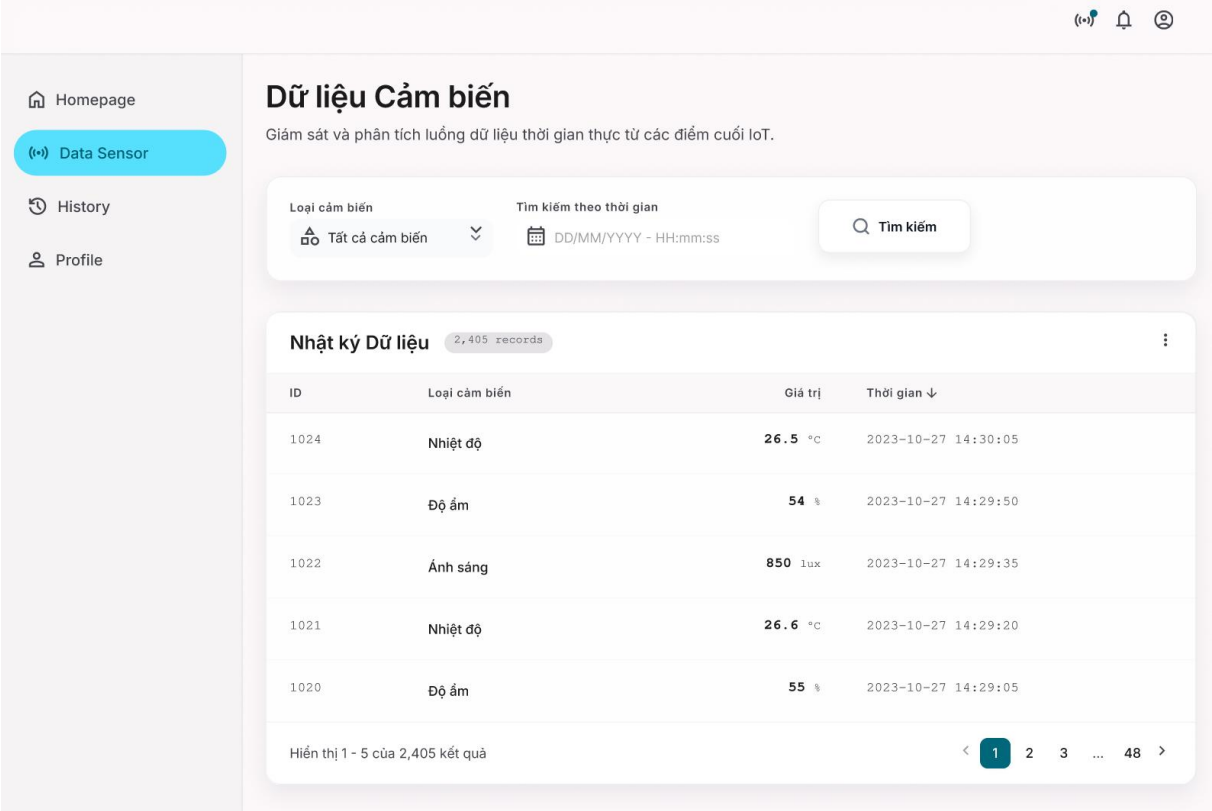
Response	{
	"id":2,
	"status":"PENDING"
	“timeoutSeconds” : 5
	}

3.4.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
id	action_history	id
device_id	action_history	device_id
action	action_history	action
status	action_history	status
created_at	action_history	created_at
Trạng thái thiết bị	devices	status
Thời điểm cập nhật trạng thái	devices	updated_at

3.5. Use case: Xem lịch xử giá trị cảm biến

3.5.1. Giao diện

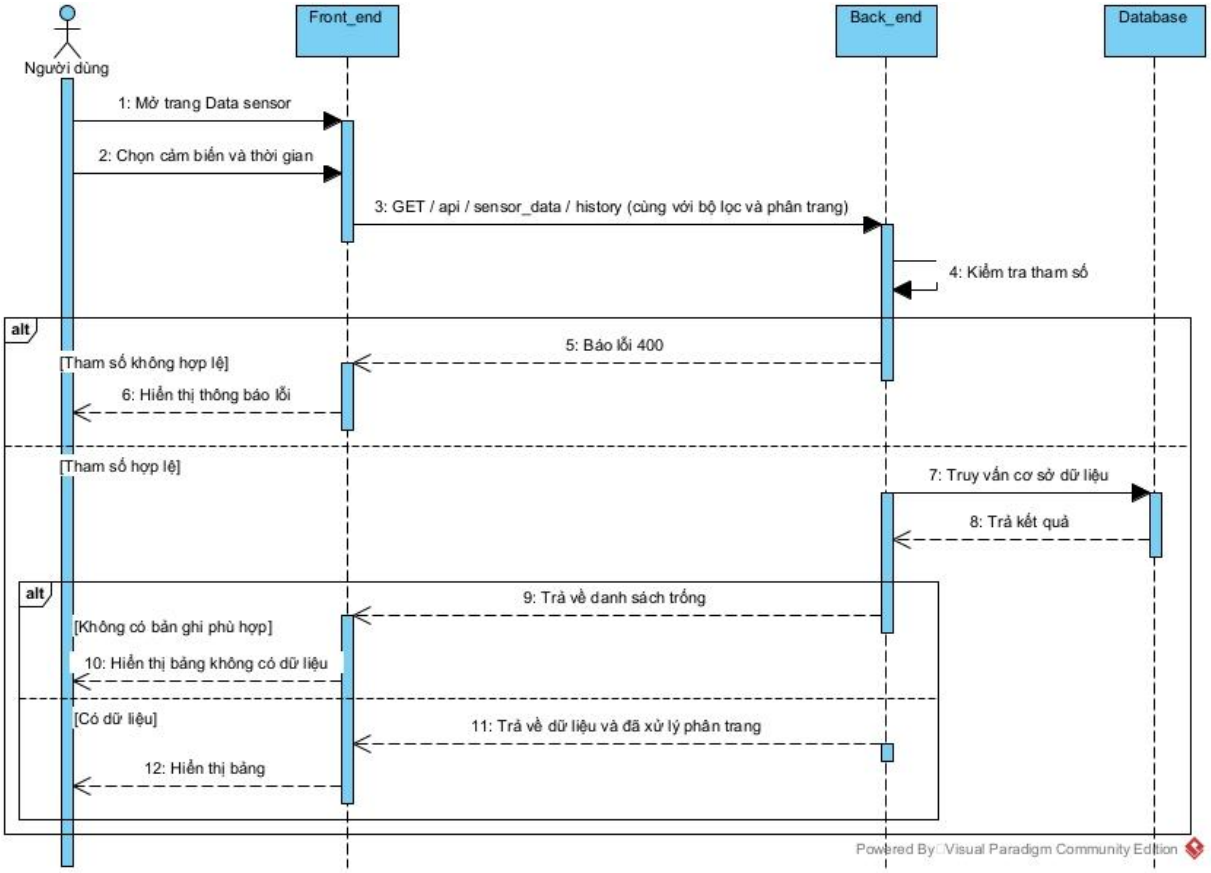


Hình 3.9. Data Sensor– lịch sử dữ liệu cảm biến

3.5.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Xem lịch sử giá trị cảm biến
Actor	Người dùng đã được xác thực
Tiền điều kiện	Người dùng đã đăng nhập và truy cập được màn hình Sensor Data.
Hậu điều kiện	Bảng hiển thị đúng dữ liệu theo điều kiện lọc, thứ tự và trang được lựa chọn.
Luồng thực hiện	Bước 1: Người dùng mở màn hình Sensor Data. Bước 2: Người dùng lựa chọn cảm biến, khoảng thời gian. Bước 3: JavaScript gửi: GET /api/sensor-data/history kèm các tham số lọc và phân trang. Bước 4: Spring Boot kiểm tra các tham số. Bước 5: Spring Data JPA thực hiện truy vấn MySQL. Bước 6: Backend trả kết quả dạng JSON có phân trang. Bước 7: JavaScript hiển thị dữ liệu trên bảng.
Luồng ngoại lệ	• Tham số không hợp lệ: backend trả HTTP 400. • Không có dữ liệu phù hợp: bảng hiển thị danh sách rỗng. • Lỗi cơ sở dữ liệu: backend trả HTTP 500 và giao diện hiển thị thông báo lỗi.

3.5.3. Sequence diagram



Hình 3.10. Sequence diagram – Xem lịch sử giá trị cảm biến

3.5.4. Mô tả API

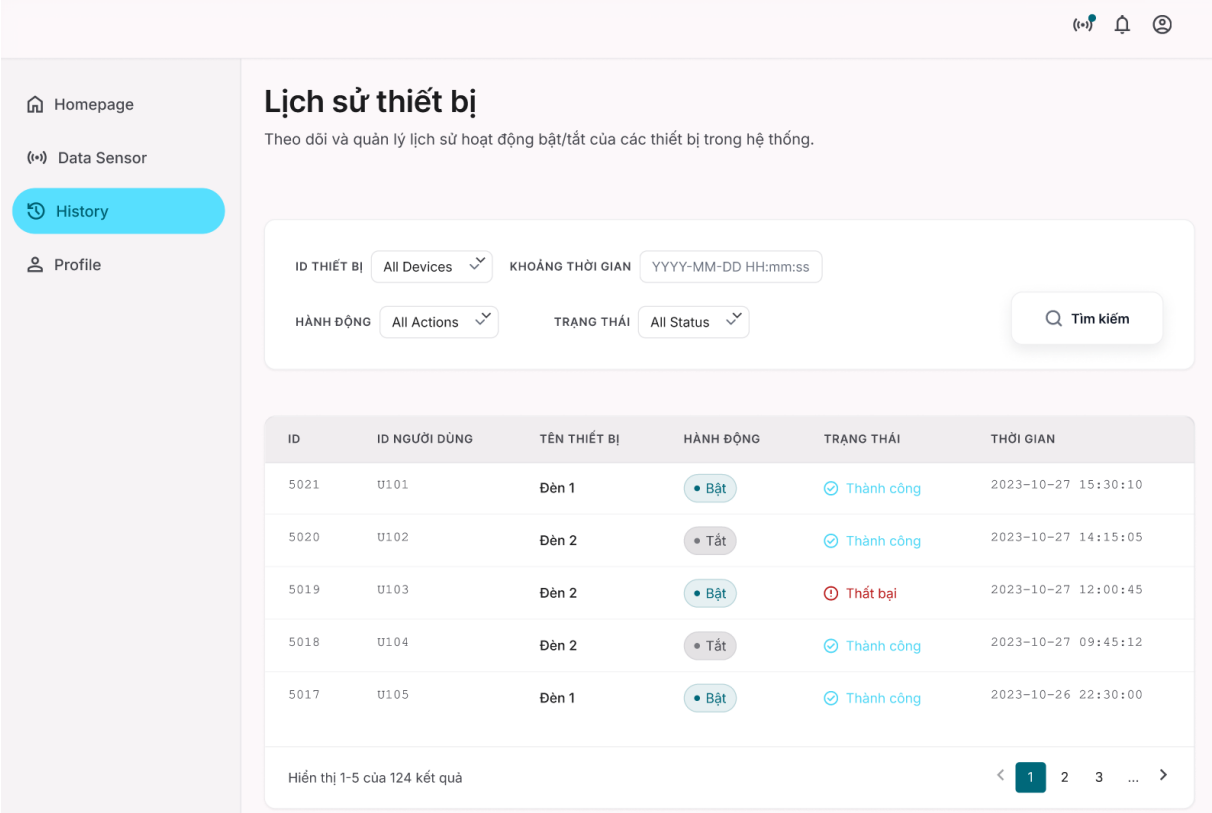
Thuộc tính	Nội dung
Tên API	Tra cứu lịch sử cảm biến
Method	GET
URL	/api/sensor-data/history
Params / Query	sensorId; from; to; page; size; sort
Body	Không có
Response	{"content": [{"id":1001,"sensorId":1,"sensorName":"Temperature","value":28.5,"createdAt":"..."}]," totalPages":5}

3.5.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
content[].id	sensor_data	id
content[].sensorId	sensor_data	sensor_id
content[].sensorName	sensors	name
content[].value	sensor_data	value
content[].createdAt	sensor_data	created_at

3.6. Use case: Xem lịch sử điều khiển thiết bị

3.6.1. Giao diện



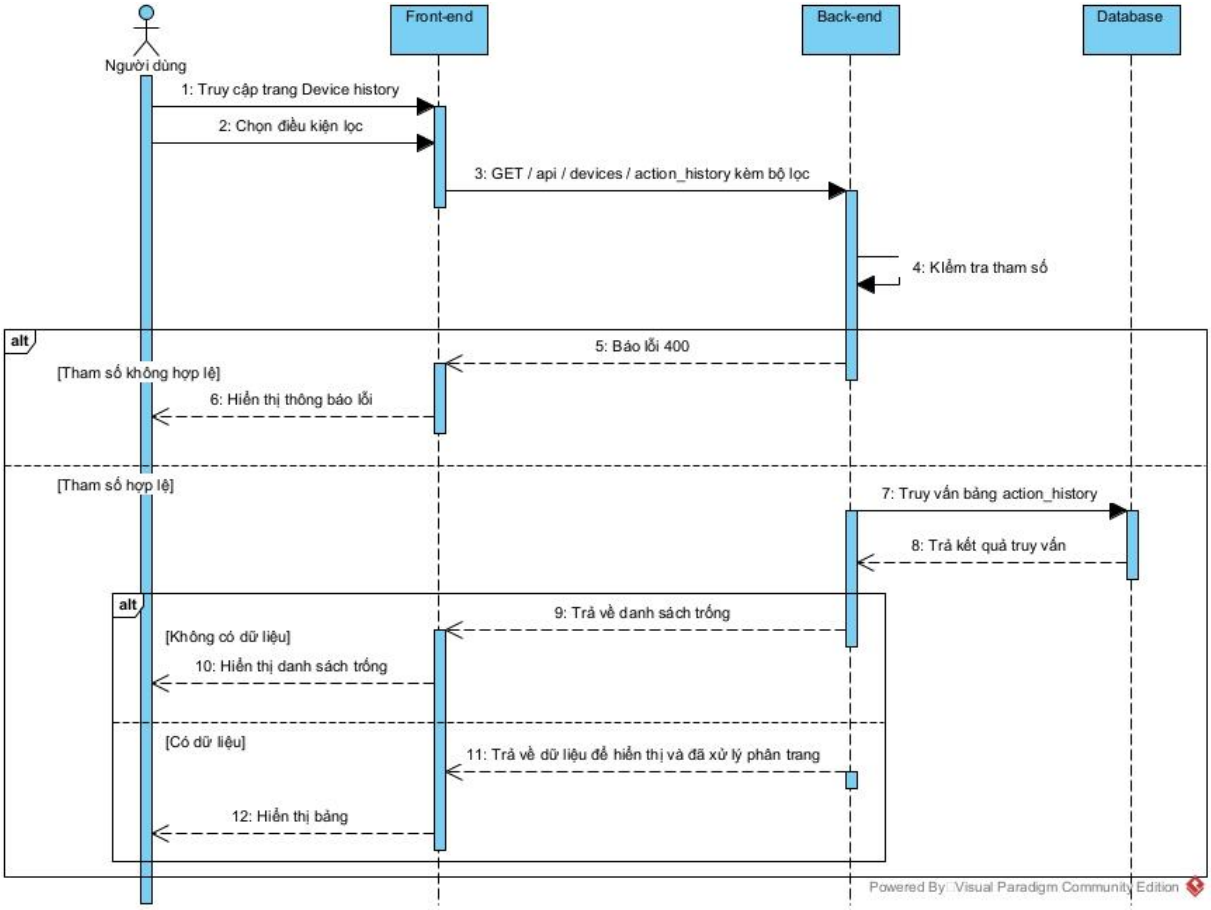
Hình 3.11. Device History – lịch sử điều khiển thiết bị

3.6.2. Đặc tả use case

Thuộc tính	Nội dung
Tên use case	Xem lịch sử điều khiển thiết bị
Actor	Người dùng đã được xác thực
Tiền điều kiện	Người dùng đã đăng nhập và truy cập được màn hình Device History.
Hậu điều kiện	Bảng hiển thị đúng lịch sử điều khiển theo bộ lọc và phân trang.
Luồng thực hiện	Bước 1: Người dùng mở màn hình Device History. Bước 2: Người dùng lựa chọn thiết bị, hành động, trạng thái hoặc khoảng thời gian. Bước 3: JavaScript gửi: GET /api/devices/action-history kèm điều kiện lọc. Bước 4: Spring Boot kiểm tra tham số. Bước 5: Spring Data JPA truy vấn bảng action_history và các bảng liên quan. Bước 6: MySQL trả kết quả. Bước 7: Backend trả dữ liệu phân trang về Web Application. Bước 8: JavaScript hiển thị lịch sử trên bảng.
Luồng ngoại lệ	• Không có dữ liệu phù hợp: trả danh sách rỗng. • Tham số không hợp lệ: API trả HTTP 400.

	• Lỗi cơ sở dữ liệu: API trả HTTP 500 và giao diện yêu cầu người dùng thử lại.

3.6.3. Sequence diagram



Hình 3.12. Sequence diagram – Xem lịch sử điều khiển thiết bị

3.6.4. Mô tả API

Thuộc tính	Nội dung
Tên API	Tra cứu lịch sử điều khiển
Method	GET
URL	/api/devices/action-history
Params / Query	deviceId; action; status; from; to; page; size; sort
Body	Không có
Response	{"content":[{"id":125,"deviceName":"Lamp 1","action":"ON","status":"SUCCESS","createdAt":"..."}],"totalPages":4}

3.6.5. Mapping dữ liệu API – Database

Trường dữ liệu	Bảng	Cột
content[].id	action_history	id
content[].deviceName	devices	name
content[].action	action_history	action
content[].status	action_history	status
content[].userId	action_history	user_id
content[].createdAt	action_history	created_at

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày kết quả triển khai mô hình giám sát và điều khiển hai đèn bằng ESP32, các màn hình của ứng dụng Web và kết quả kiểm thử. Phần cứng đã được vận hành với cảm biến nhiệt độ, độ ẩm, ánh sáng và hai LED. Đối với chức năng mới dùng JWT và WebSocket/STOMP, kết quả tự động được kiểm chứng trên máy chủ thử nghiệm với cơ sở dữ liệu H2 và cổng MQTT mô phỏng; các bằng chứng này được tách riêng với quan sát trên phần cứng thật.

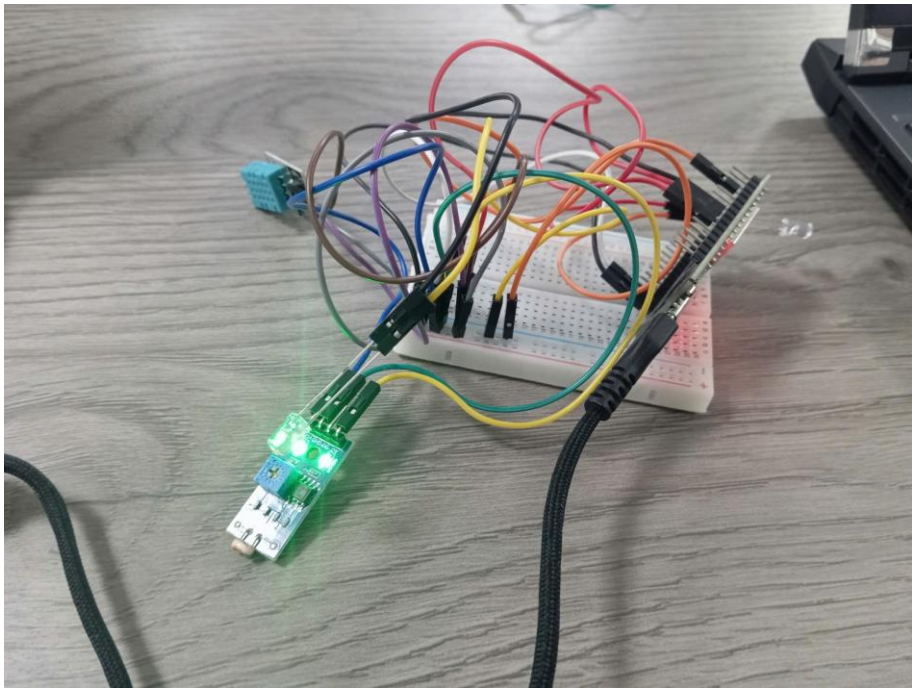
4.1. Kết quả triển khai hệ thống

4.1.1. Mô hình phần cứng và chương trình ESP32

Mô hình dùng một ESP32 với hai LED rời, một cảm biến DHT11 và một đầu đo ánh sáng nối ngõ analog. Mỗi LED có một điện trở giới hạn dòng mắc nối tiếp trong nhánh riêng. Firmware tổ chức hai đèn bằng khóa lamp1 và lamp2; khi bổ sung thiết bị trong tương lai, mã và cấu hình của thiết bị mới sẽ được khai báo tại thời điểm bổ sung. Ở phạm vi báo cáo này, hệ thống phục vụ đúng hai đèn.

Bảng 4.1. Kết nối phần cứng trong mô hình

Thiết bị	Chân ESP32	Vai trò	Dữ liệu hoặc trạng thái
LED 1	GPIO25	Ngõ ra số	lamp1: ON/OFF
LED 2	GPIO26	Ngõ ra số	lamp2: ON/OFF
DHT11	GPIO27	Ngõ vào số	Nhiệt độ (°C), độ ẩm (%)
Đầu đo ánh sáng	GPIO34	Ngõ vào ADC	Giá trị 0–4095 ADC



Hình 4.1. Mô hình ESP32, cảm biến và hai đèn trong báo cáo gốc

ESP32 đọc DHT11 và ADC khoảng mỗi 2 giây. Nếu lần đọc DHT11 không hợp lệ, firmware bỏ qua lần gửi đó. Giá trị ánh sáng được biểu diễn theo thang ADC 0–4095, chưa được hiệu chuẩn thành đơn vị lux. Việc ghi nhận một lệnh SUCCESS chỉ cho biết ESP32 đã xử lý GPIO và gửi xác nhận; hệ thống không có cảm biến độc lập để chứng minh LED phát sáng.

4.1.2. Truyền thông, lưu trữ và máy chủ

ESP32 dùng định danh device01 và kết nối Mosquitto qua MQTT (cấu hình mặc định cổng 1001). Bản tin số đo đi qua iot/device01/sensor. Backend gửi lệnh điều khiển lên iot/device01/control, gồm requestId, deviceKey và action. ESP32 trả phản hồi qua iot/device01/ack; trạng thái riêng của từng đèn được phát ở iot/device01/status/lamp1 và iot/device01/status/lamp2. Chủ đề iot/device01/availability thông báo ONLINE/OFFLINE. Trạng thái đèn và kết nối được giữ lại trên broker để đồng bộ khi kết nối lại; phản hồi lệnh không được giữ lại.

Spring Boot tiếp nhận MQTT, ghi số đo và lịch sử lệnh vào MySQL iot_db. Các bảng chính là users, sensors, sensor_data, devices và action_history. requestId là khóa ghép một lệnh Web với phản hồi từ ESP32. Backend chỉ coi lệnh hoàn tất sau khi nhận phản hồi tương ứng hoặc hết thời gian chờ; khi TIMEOUT, trạng thái ON/OFF cuối cùng đã được xác nhận vẫn được giữ nguyên.

Bảng 4.2. Thành phần phần mềm đã triển khai

Thành phần	Kết quả triển khai	Điểm kiểm chứng
ESP32	Đọc ba đại lượng, điều khiển hai GPIO và gửi ACK	Firmware sketch_sep11a.ino; lần chạy phần cứng
Mosquitto	Chuyển bản tin cảm biến, lệnh, ACK và trạng thái	Luồng MQTT theo device01
MySQL	Lưu thiết bị, cảm biến, số đo, người dùng, lịch sử	Các bảng trong iot_db
Spring Boot	REST, JWT Bearer, STOMP/WebSocket	Bộ thử HTTP/STOMP
Web	Đăng nhập, Dashboard, tra cứu và hồ sơ	Ảnh giao diện và thử nghiệm trình duyệt

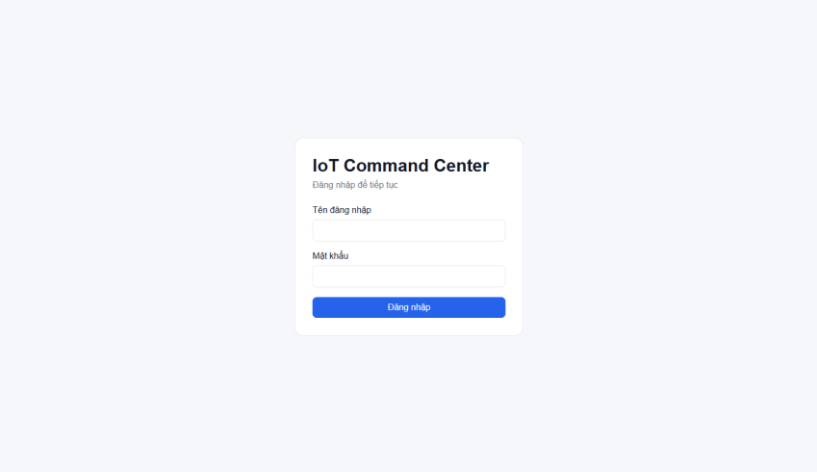
Máy chủ cung cấp giao diện HTML/CSS/JavaScript cùng nguồn gốc với API tại cổng 8080. Sau khi đăng nhập, client gửi JWT Bearer cho REST và trong khung STOMP CONNECT. Backend đẩy thay đổi qua /topic/sensor-data, /topic/devices, /topic/availability và kênh riêng /user/queue/commands. Giao diện không cần tải lại trang để nhận số đo hoặc kết quả lệnh mới.

4.2. Kết quả giao diện và chức năng

Các hình trong mục này thể hiện mã giao diện hiện hành. Hình Dashboard được lấy từ ảnh chụp phiên chạy với ESP32 trong báo cáo gốc. Ảnh đăng nhập, dữ liệu cảm biến, lịch sử và hồ sơ được dựng từ giao diện hiện hành với dữ liệu minh họa trong môi trường thử, không dùng để khẳng định số đo hoặc lệnh thực tế tại thời điểm chụp.

a. Màn hình đăng nhập

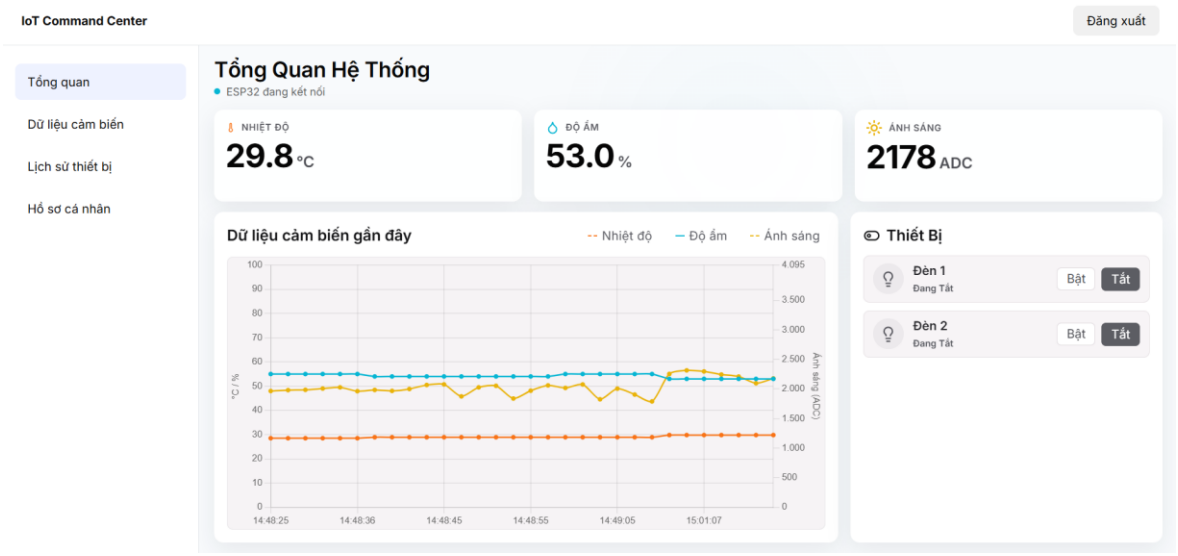
Người dùng nhập tên đăng nhập và mật khẩu; frontend gửi JSON tới POST /api/auth/login. Khi xác thực thành công, backend trả JWT cùng thời hạn. Frontend lưu JWT trong sessionStorage của tab, gắn Bearer cho những API được bảo vệ và chuyển về trang đăng nhập khi token hết hạn hoặc nhận HTTP 401. Nút Đăng xuất xóa token trong tab và ngắt kết nối WebSocket.



Hình 4.2. Màn hình đăng nhập của giao diện hiện hành

b. Dashboard và điều khiển hai đèn

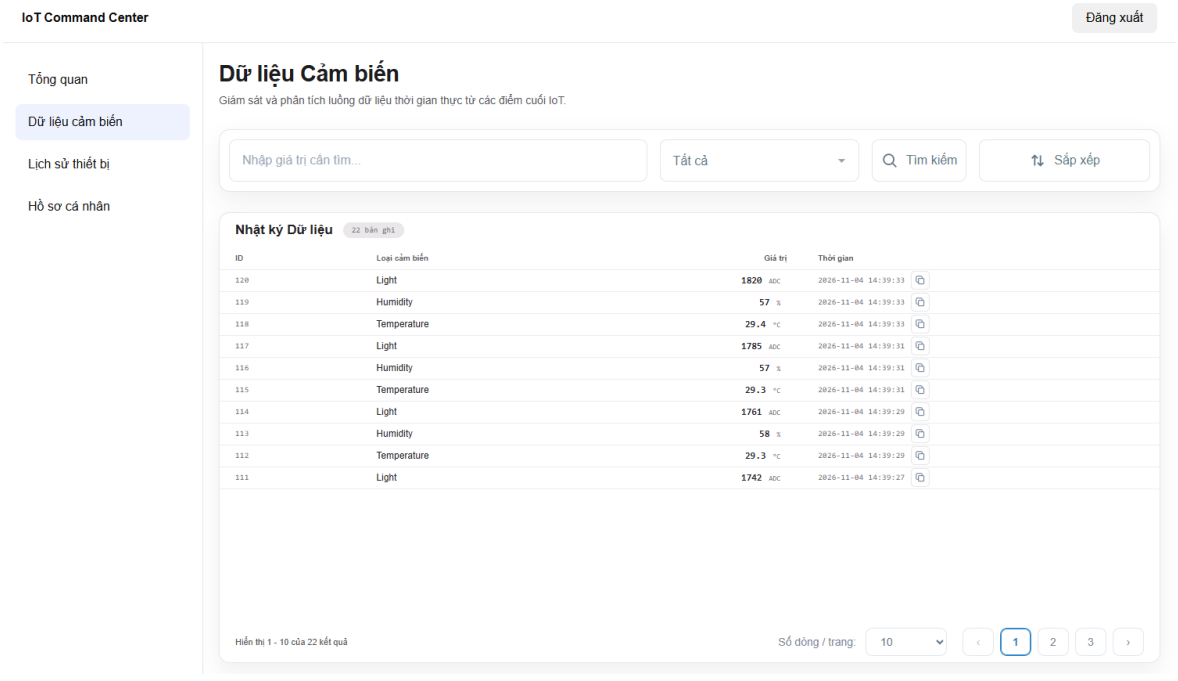
Dashboard hiển thị nhiệt độ, độ ẩm, ánh sáng, trạng thái kết nối ESP32, hai đèn và biểu đồ gần nhất. Ở thời điểm ảnh nguồn được chụp, giao diện hiển thị 29,8 °C, 53% và 2178 ADC; đây là một ảnh chụp đơn lẻ, không phải phép đo độ chính xác. Nút bật/tắt tạo lệnh HTTP 202 ở trạng thái PENDING. Kết quả cuối cùng được nhận qua STOMP hoặc tra cứu bằng requestId, vì vậy giao diện phân biệt việc gửi lệnh với việc ESP32 xác nhận.



Hình 4.3. Dashboard trong ảnh chụp phiên chạy thực tế của báo cáo gốc

c. Màn hình dữ liệu cảm biến

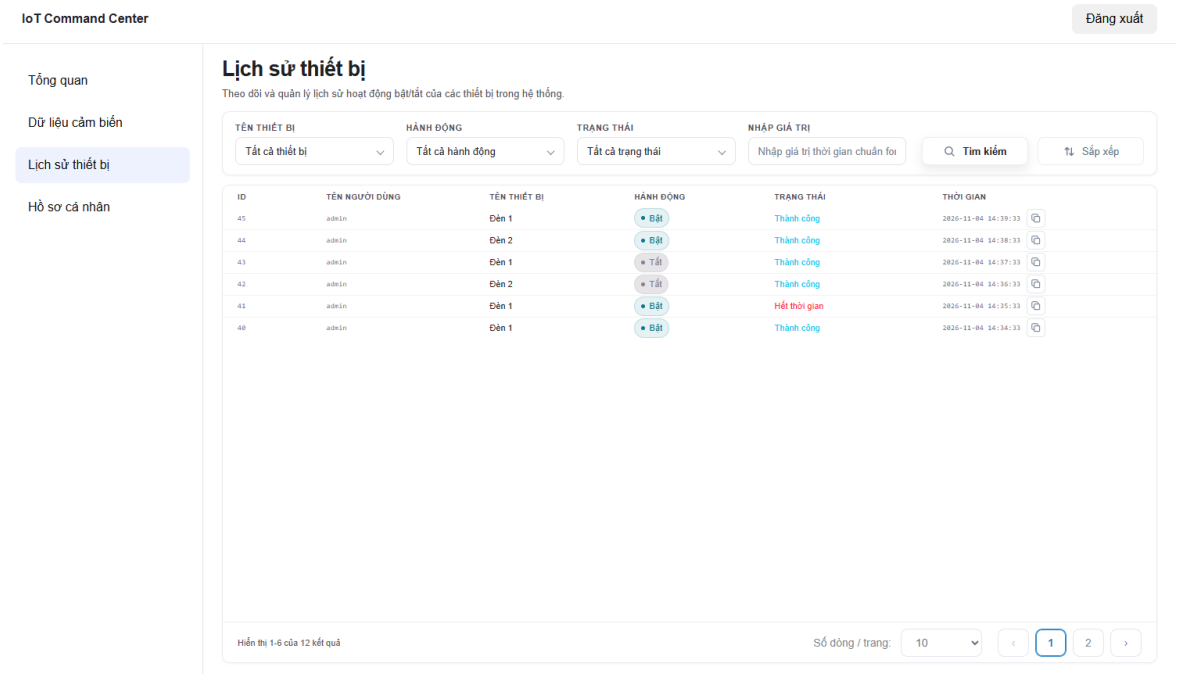
Màn hình hiển thị các bản ghi theo trang với ID, loại cảm biến, giá trị, đơn vị và thời gian. Người dùng có thể lọc theo cảm biến, giá trị chính xác hoặc khoảng thời gian tương ứng với ngày, phút hay giây đã nhập; kết quả chỉ gồm các bản ghi trong khoảng này. Mỗi thời điểm có nút sao chép. Dữ liệu ảnh dưới đây là dữ liệu minh họa để xem bố cục, gồm ba loại cảm biến của mô hình.



Hình 4.4. Màn hình dữ liệu cảm biến với dữ liệu minh họa

d. Màn hình lịch sử điều khiển

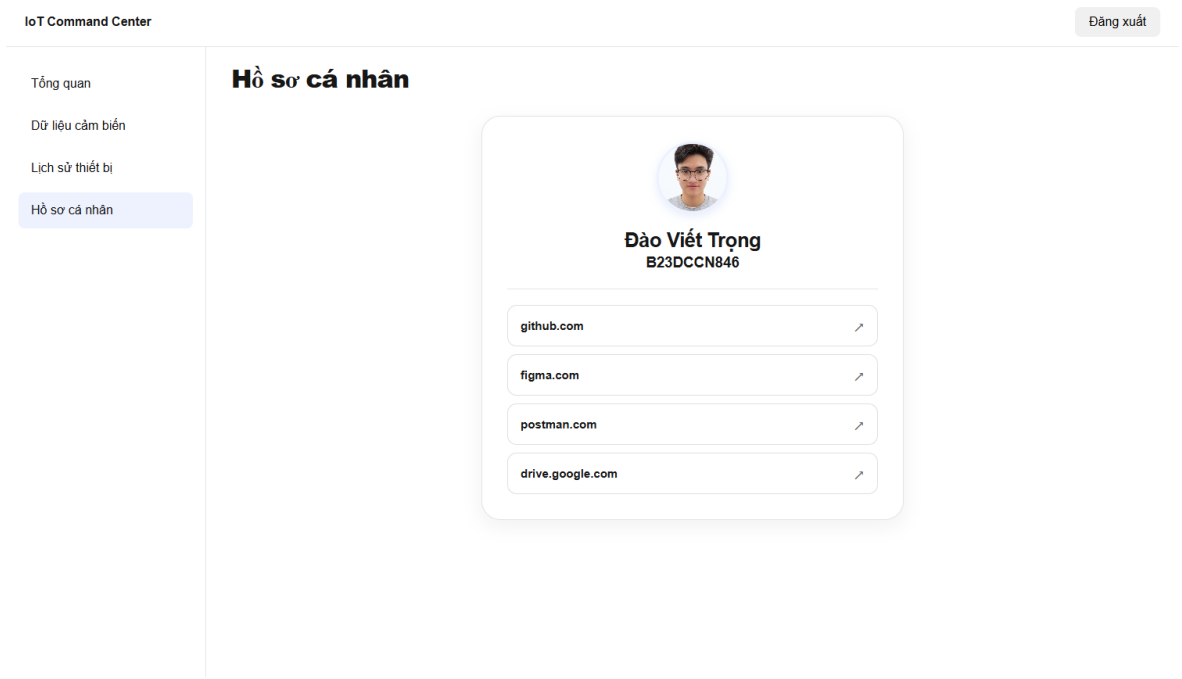
Lịch sử ghi người thao tác, thiết bị, hành động ON/OFF, kết quả và thời gian. Bộ lọc hỗ trợ thiết bị, hành động, trạng thái và thời gian; bảng có phân trang, sắp xếp và sao chép thời điểm. Các trạng thái PENDING, SUCCESS, FAILED và TIMEOUT phản ánh vòng đời lệnh, trong đó TIMEOUT không tự đổi trạng thái đèn đã xác nhận trước đó.



Hình 4.5. Màn hình lịch sử điều khiển với dữ liệu minh họa

e. Màn hình hồ sơ cá nhân

Trang hồ sơ trình bày thông tin sinh viên và các liên kết tài liệu, thiết kế Figma, Postman. Liên kết Figma và Postman dẫn tới tài nguyên đã được cung cấp trong dự án; mục GitHub và Google Drive trên giao diện hiện vẫn là liên kết chung, cần thay bằng địa chỉ dự án cụ thể khi công bố. Trang này là phần hiển thị thông tin, không phải chức năng chỉnh sửa hồ sơ trong cơ sở dữ liệu.



Hình 4.6. Màn hình hồ sơ cá nhân của giao diện hiện hành

4.3. Kết quả kiểm thử

Kiểm thử tự động được chạy trên ứng dụng Spring Boot ở cổng ngẫu nhiên, cơ sở dữ liệu H2 ở chế độ thử nghiệm và MQTT gateway mô phỏng. Kết quả gần nhất: 16 phép thử đạt, không có lỗi hoặc thất bại; một phép thử phụ thuộc broker thật được bỏ qua theo điều kiện môi trường. Bài thử trình duyệt Edge kiểm tra đường đi đăng nhập, Bearer cho REST, STOMP CONNECT, lọc thời gian, tải lại trang và đăng xuất. Các phép thử này xác minh hành vi phần mềm; chúng không thay thế đo kiểm phần cứng sau lần chuyển sang JWT.

Bảng 4.3. Đối chiếu chức năng API và kết quả thử

Chức năng	Đầu vào hoặc tình huống	Kết quả quan sát
Đăng nhập	POST /api/auth/login: thiếu mật khẩu; sai mật khẩu; hợp lệ	Lần lượt HTTP 400, 401, 200; trả JWT Bearer, không tạo cookie phiên
Bảo vệ REST	GET /api/devices thiếu, sai hoặc hết hạn JWT	HTTP 401; JWT hợp lệ đọc được hai thiết bị
Dữ liệu cảm biến	Bản tin MQTT có nhiệt độ, độ ẩm, ánh sáng	Ghi ba bản ghi; latest, chart và history phản ánh dữ liệu; STOMP đẩy sau commit
Trạng thái thiết bị	ONLINE và trạng thái lamp1 OFF	GET /api/devices/{deviceId}/status trả OFF đã xác nhận
Điều khiển	POST /api/devices/{deviceId}/control với ON	HTTP 202, PENDING và requestId; ACK hợp lệ đổi kết quả thành SUCCESS
Tra cứu lệnh	GET /api/commands/{requestId} bằng tài khoản khác	HTTP 404, không lộ lệnh của người dùng trước
Lịch sử điều khiển	GET /api/devices/action-history sau ACK	Bản ghi có trạng thái SUCCESS

Chức năng	Đầu vào hoặc tình huống	Kết quả quan sát
Hết thời gian chờ	Gửi OFF nhưng không có ACK	TIMEOUT sau thời hạn; trạng thái ON đã xác nhận không bị ghi đè
Bảo vệ STOMP	CONNECT thiếu/hết hạn JWT; client SEND trái phép	Bị từ chối; kết nối đã xác thực đóng khi JWT hết hạn

Các API GET /api/sensor-data/latest, /api/sensor-data/chart, /api/sensor-data/history và GET /api/devices/action-history có đường dẫn đúng với mã hiện hành. Tham số phân trang dùng page bắt đầu từ 0, size từ 1 đến 100; biểu đồ giới hạn số điểm mỗi cảm biến. Các truy vấn từ thời gian đến thời gian bao gồm cả hai mốc. Kết quả rỗng của danh sách là HTTP 200 với nội dung rỗng, không phải lỗi 404.

Trong lần thử phần cứng trước đó, người thực hiện đã xác nhận ESP32 gửi được số đo vào iot_db và hai đèn có thể bật sáng sau khi thay LED hỏng, mắc điện trở nối tiếp. Bằng chứng này thuộc quá trình tích hợp thực tế trước khi đổi sang JWT; Chương 4 không suy diễn rằng toàn bộ kịch bản JWT đã được chạy lại với broker, cơ sở dữ liệu và ESP32 thật.

4.4. Đánh giá kết quả

Bảng 4.4. Mức độ đáp ứng yêu cầu và giới hạn

Yêu cầu	Kết quả đạt được	Giới hạn cần lưu ý
Giám sát ba đại lượng	ESP32 phát bản tin; backend lưu và hiển thị theo thời gian	Ánh sáng ở thang ADC, chưa hiệu chuẩn lux
Điều khiển hai đèn	lamp1 và lamp2 có lệnh, ACK, trạng thái và lịch sử riêng	SUCCESS xác nhận GPIO, không đo ánh sáng LED
Cập nhật thời gian thực	Backend đẩy sự kiện qua WebSocket/STOMP sau khi ghi dữ liệu	Thử tự động trong môi trường giả lập MQTT
Tra cứu dữ liệu	Lọc, sắp xếp, phân trang, sao chép thời gian	Chưa đánh giá tải lớn và vận hành dài hạn
Bảo vệ hệ thống	REST và STOMP dùng JWT Bearer; API đăng nhập công khai	Chưa có refresh token hoặc thu hồi token phía máy chủ

Kết quả cho thấy mô hình đã đáp ứng các chức năng cốt lõi của đề tài trong phạm vi hai đèn: thu thập số đo, lưu trữ, quan sát qua Web, gửi lệnh và theo dõi xác nhận. Phần thử tự động chứng minh các luồng API, JWT và STOMP hoạt động trong môi trường thử, còn lần chạy phần cứng chứng minh đường truyền dữ liệu và thao tác đèn ở cấu hình đã kiểm tra. Để đánh giá độ tin cậy thực tế sau khi đổi xác thực, cần chạy lại kiểm thử đầu cuối với broker, MySQL và ESP32 thật; nếu muốn phát hiện LED cháy, phải bổ sung phần cứng đo dòng hoặc ánh sáng riêng.